PROCESOS

Sistemas Distribuidos

Alvaro Ospina Sanjuan
Universidad Pontificia Bolivariana
Medellín 2026

Proceso

- Es un programa en ejecución - un flujo de control – Entidad Activa
- El sistema operativo proporciona esa abstracción.
- Cada proceso piensa que está solo en la máquina, que tiene su propia CPU y su propia memoria. (Consume recursos limitados del Hardware)
- Los procesos tienen distintos datos asociados:
  - Código del programa
  - Pila
  - Datos

Carga de Procesos

- Fuente → Compilación → Enlazado → Binario (ejecutable).
- El archivo binario contiene la información para crear un proceso, organizadas en secciones. Hay distintos formatos: ELF o Extensible Linking Format, PE (Windows), a.out (Unix antiguos), etc. (compilación cruzada)
- El binario es simplemente una receta para crear un proceso para ejecutar el programa.
- En su cabecera se indica la arquitectura, el tamaño/**offset** de las tablas y secciones, y el punto de entrada (dirección para comenzar a ejecutar).

Carga de Procesos (1/2)

- Cuando se ejecuta un programa, el cargador (que forma parte del kernel), hace lo siguiente:
- 1. Comprueba si se puede ejecutar (permisos, etc.).
- 2. Comprueba el tipo de ejecutable (interpretado o binario). Si el archivo empieza por los caracteres #!, es interpretado: se ejecutará el interprete indicado a continuación, con la ruta de este archivo como argumento.
- 3. Copia las secciones del ELF a memoria, que se organiza en segmentos.
- 4. Copia los argumentos del programa y variables de entorno (argc, argv, envp) a la pila del proceso.

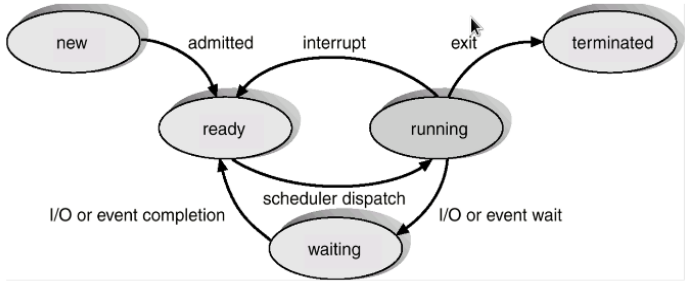
Carga de Procesos (2/2)

- 5. Inicializa el contexto del proceso (registros, etc.) y sus atributos: se asigna PID (Process ID), etc.
- 6. Pone el proceso a ejecutar, comenzará por su punto de entrada.

Tipos de Procesos

- Sistemas procesamiento por lotes —— jobs
- Sistemas tiempo compartido ——— tasks
- Sistemas monousuario ——————— program

Estados de un Proceso



1. Se bloquea en espera de datos
2. Los datos están disponibles
3. Se asigna a la CPU por el fijador del kernel
4. Se le quita la CPU

En ejecución, Listo o preparado, terminado, Nuevo, Espera

Implantación

- **Tabla de procesos:** Observarla en linux con "??"

- **Demonios:** Procesos creados por el SO para ejecución permanente. En linux terminan en "d"

- **Atributos de un proceso:**
  - Todos los procesos tiene un PID o Process ID, porque?
  - El PID es único
  - Todo los procesos tiene un proceso padre que es el proceso que lo inició
    - Excepto el proceso 0 el cual es iniciado por el kernel

Llamadas para Procesos en C (linux)

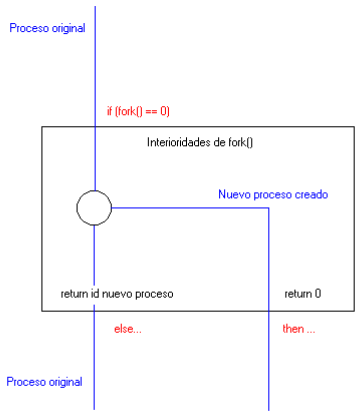
- ***pid_t fork(void):*** Crea un proceso, donde el hijo es copia exacta del padre. La función retorna al proceso hijo "0" y al proceso padre el pid del hijo.

- ***int execv(const char *path, char * argv[]):*** Reemplaza la imagen de un proceso.

- ***pid_t wait(int * stat_loc):*** Suspende al padre hasta que llegue el hijo o una señal.

- ***void _exit(int status):*** Termina el proceso y lo envía donde el padre.

Estados de un Proceso



El proceso original sigue por la parte del **else** y *el nuevo proceso por el then*. De esta manera, el proceso original, por ejemplo, podría seguir atendiendo nuevos clientes que quieran conectarse a nuestro programa por un socket, mientras que el proceso hijo podría atender a un cliente que acaba de conectarse y que es el que ha provocado que lancemos el fork().

Proceso

- Hay que tener en cuenta que se ha duplicado todo el espacio de memoria. Por ello, ambos procesos tienen todas las variables repetidas, pero distintas. Si el proceso original toca la variable "contador", el proceso hijo no verá reflejado el cambio en su versión de "contador".
- si antes del fork() tenemos, por ejemplo, un flujo abierto (un flujo a archivo, un socket o cualquier otra cosa), después del fork() ambos procesos tendrán abierto el mismo flujo y ambos podrán escribir en él. Es más, uno de los procesos puede cerrar el flujo mientras que el otro lo puede seguir teniendo abierto.

Proceso

- Proceso tiene su propia memoria, es imposible en Linux que un proceso se “meta” en la memoria de otro.